ONNX Runtime Server: The ONNX Runtime Server is a server that provides TCP and HTTP/HTTPS REST APIs for ONNX inference.

MIT license

153 stars 11 forks 2 watching Branches Activity
Tags

Public repository

main 24 Branches 14 Tags Go to file t Go to file Add file Code

kibae feat: update swagger (#95) ✓ 5e4ea8f · last month

.github	Release/1.21.0 (#94)	last month
cmake	feat: Windows support (#80)	3 months ago
deploy	Release/1.21.0 (#94)	last month
docs	feat: update swagger (#95)	last month
src	Release/1.21.0 (#94)	last month
test	feat: #66 Update default model path (#72)	4 months ago
.clang-format	feat: --prepare-model option (#10)	2 years ago
.gitignore	ci: docker	2 years ago
CMakeLists.txt	fix: cmake min version	2 years ago
LICENSE	Create LICENSE	2 years ago
README.md	Release/1.21.0 (#94)	last month
download-onnxruntime-linux.sh	chore: remove unnecessary	2 years ago

ONNX Runtime Server

ONNX Runtime v1.21.0 | CMake on Linux passing | CMake on MacOS passing | CMake on Windows passing | CodeQL passing | license MIT

- ONNX: Open Neural Network Exchange
- **The ONNX Runtime Server is a server that provides TCP and HTTP/HTTPS REST APIs for ONNX inference.**
- ONNX Runtime Server aims to provide simple, high-performance ML inference and a good developer experience.
  - If you have exported ML models trained in various environments as ONNX files, you can provide inference APIs without writing additional code or metadata. Just place the ONNX files into the directory structure.
  - Each ONNX session, you can choose to use CPU or CUDA.
  - Analyze the input/output of ONNX models to provide type/shape information for your collaborators.
  - Built-in Swagger API documentation makes it easy for collaborators to test ML models through the API. (API example)
  - Ready-to-run Docker images. No build required.

- Build ONNX Runtime Server
  - Requirements
    - Install ONNX Runtime
    - Install dependencies
  - Compile and Install

Build ONNX Runtime Server

Requirements

- ONNX Runtime
- Boost
- CMake, pkg-config
- CUDA(*optional, for Nvidia GPU support*)
- OpenSSL(*optional, for HTTPS*)

Install ONNX Runtime

Linux

- Use `download-onnxruntime-linux.sh` script
  - This script downloads the latest version of the binary and install to `/usr/local/onnxruntime` .
  - Also, add `/usr/local/onnxruntime/lib` to `/etc/ld.so.conf.d/onnxruntime.conf` and run `ldconfig` .
- Or manually download binary from ONNX Runtime Releases.

Mac OS

```
brew install onnxruntime
```

Install dependencies

Ubuntu/Debian

```
sudo apt install cmake pkg-config libboost-all-dev libssl-dev
```

(optional) CUDA support (CUDA 12.x, cuDNN 9.x)

- Follow the instructions below to install the CUDA Toolkit and cuDNN.
  - CUDA Toolkit Installation Guide
  - CUDA Download for Ubuntu

```
sudo apt install cuda-toolkit-12 libcudnn9-dev-cuda-12
# optional, for Nvidia GPU support with Docker
sudo apt install nvidia-container-toolkit
```

Mac OS

```
brew install cmake boost openssl
```

Compile and Install

```
cmake -B build -S . -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
sudo cmake --install build --prefix /usr/local/onnxruntime-server
```

Install via a package manager

OS	Method	Command
Arch Linux	AUR	`yay -S onnxruntime-server`

Run the server

- **You must enter the path option(`--model-dir`) where the models are located.**
  - The onnx model files must be located in the following path: `${model_dir}/${model_name}/${model_version}/model.onnx` or `${model_dir}/${model_name}/${model_version}.onnx`

Files in `--model-dir`	Create session request body	Get/Execute session API URL path (after created)
`model_name/model_version/model.onnx` or `model_name/model_version.onnx`	`{"model":"model_name", "version":"model_version"}`	`/api/sessions/model_name/model_version`
`sample/v1/model.onnx` or `sample/v1.onnx`	`{"model":"sample", "version":"v1"}`	`/api/sessions/sample/v1`
`sample/v2/model.onnx` or `sample/v2.onnx`	`{"model":"sample", "version":"v2"}`	`/api/sessions/sample/v2`
`other/20200101/model.onnx` or `other/20200101.onnx`	`{"model":"other", "version":"20200101"}`	`/api/sessions/other/20200101`

- **You need to enable one of the following backends: TCP, HTTP, or HTTPS.**
  - If you want to use TCP, you must specify the `--tcp-port` option.
  - If you want to use HTTP, you must specify the `--http-port` option.
  - If you want to use HTTPS, you must specify the `--https-port` , `--https-cert` and `--https-key` options.
  - If you want to use Swagger, you must specify the `--swagger-url-path` option.
- Use the `-h` , `--help` option to see a full list of options.
- **All options can be set as environment variables.** This can be useful when operating in a container like Docker.
  - Normally, command-line options are prioritized over environment variables, but if the `ONNX_SERVER_CONFIG_PRIORITY=env` environment variable exists, environment variables have higher priority. Within a Docker image, environment variables have higher priority.

Options

Option	Environment	Description
`--workers`	`ONNX_SERVER_WORKERS`	Worker thread pool size. Default: `4`
`--request-payload-limit`	`ONNX_SERVER_REQUEST_PAYLOAD_LIMIT`	HTTP/HTTPS request payload size limit. Default: 1024 * 1024 * 10(10MB)`
`--model-dir`	`ONNX_SERVER_MODEL_DIR`	Model directory path The onnx model files must be located in the following path: `${model_dir}/${model_name}/${model_version}/model.onnx` or `${model_dir}/${model_name}/${model_version}.onnx` Default: `models`
`--prepare-model`	`ONNX_SERVER_PREPARE_MODEL`	Pre-create some model sessions at server startup. Format as a space-separated list of `model_name:model_version` or `model_name:model_version(session_options, ...)` . Available session_options are - cuda=device_id `[ or true or false]`

Option	Environment	Description
		eg) `model1:v1 model2:v9` `model1:v1(cuda=true) model2:v9(cuda=1)`

Backend options

Option	Environment	Description
`--tcp-port`	`ONNX_SERVER_TCP_PORT`	Enable TCP backend and which port number to use.
`--http-port`	`ONNX_SERVER_HTTP_PORT`	Enable HTTP backend and which port number to use.
`--https-port`	`ONNX_SERVER_HTTPS_PORT`	Enable HTTPS backend and which port number to use.
`--https-cert`	`ONNX_SERVER_HTTPS_CERT`	SSL Certification file path for HTTPS
`--https-key`	`ONNX_SERVER_HTTPS_KEY`	SSL Private key file path for HTTPS
`--swagger-url-path`	`ONNX_SERVER_SWAGGER_URL_PATH`	Enable Swagger API document for HTTP/HTTPS backend. This value cannot start with "/api/" and "/health" If not specified, swagger document not provided. eg) /swagger or /api-docs

Log options

Option	Environment	Description
`--log-level`	`ONNX_SERVER_LOG_LEVEL`	Log level(debug, info, warn, error, fatal)
`--log-file`	`ONNX_SERVER_LOG_FILE`	Log file path. If not specified, logs will be printed to stdout.
`--access-log-file`	`ONNX_SERVER_ACCESS_LOG_FILE`	Access log file path. If not specified, logs will be printed to stdout.

Docker

- Docker hub: kibaes/onnxruntime-server
  - `1.21.0-linux-cuda12` amd64(CUDA 12.x, cuDNN 9.x)
  - `1.21.0-linux-cpu` amd64, arm64

```
DOCKER_IMAGE=kibae/onnxruntime-server:1.21.0-linux-cuda12 # or kibae/onnxruntime-server:1.21.0-linux-cpu

docker pull ${DOCKER_IMAGE}

# simple http backend
docker run --name onnxruntime_server_container -d --rm --gpus all \
  -p 80:80 \
  -v "/your_model_dir:/app/models" \
  -v "/your_log_dir:/app/logs" \
  -e "ONNX_SERVER_SWAGGER_URL_PATH=/api-docs" \
  ${DOCKER_IMAGE}
```

- More information on using Docker images can be found here.
  - https://hub.docker.com/r/kibaes/onnxruntime-server
- docker-compose.yml example is available in the repository.

API

- HTTP/HTTPS REST API
  - API documentation (Swagger) is built in. If you want the server to serve swagger, add the `--swagger-url-path=/swagger/` option at launch. This must be used with the `--http-port` or `--https-port` option.

```
./onnxruntime_server --model-dir=YOUR_MODEL_DIR --http-port=8080 --swagger-url-path=/api-docs/
```

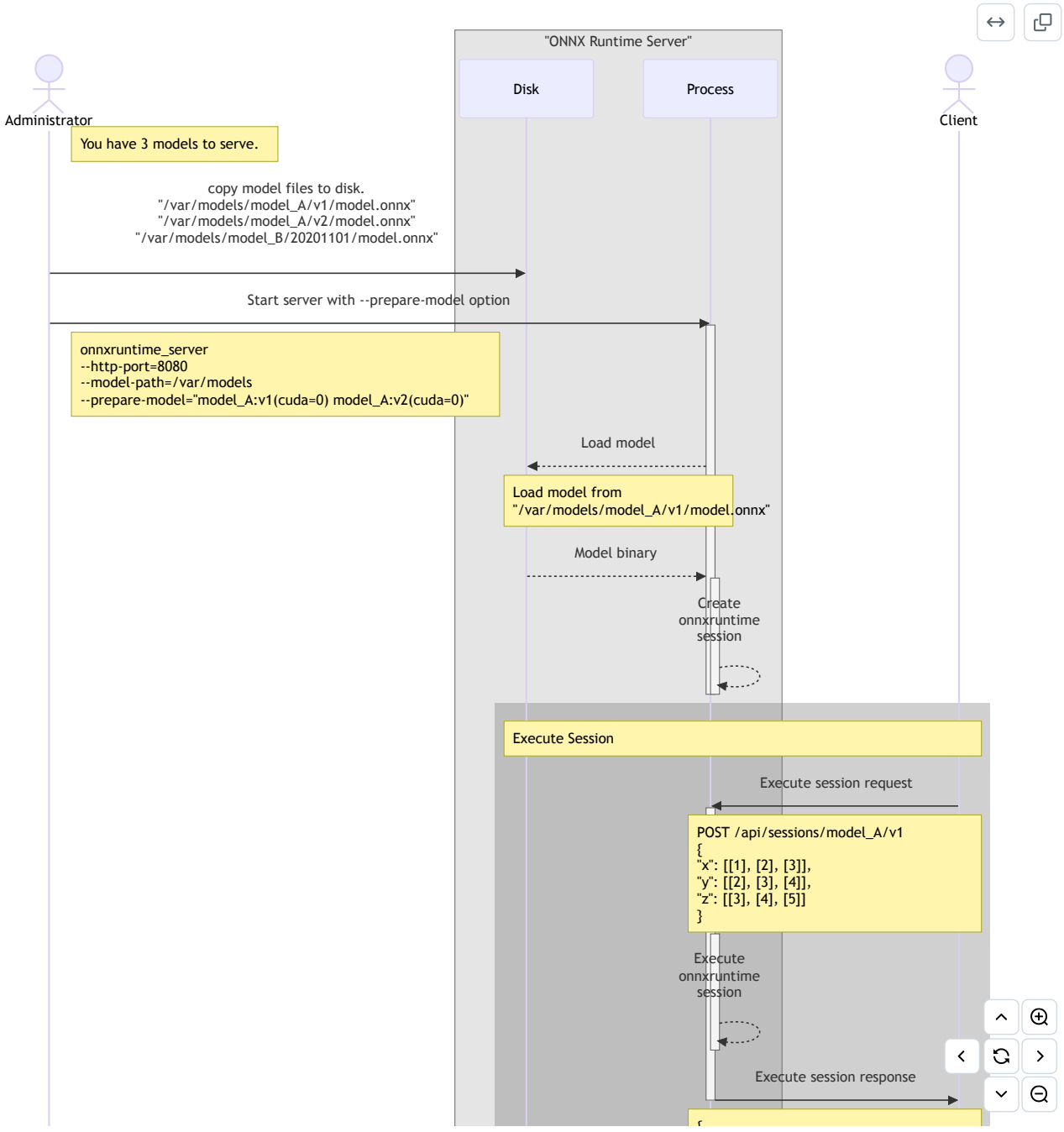
    - After running the server as above, you will be able to access the Swagger UI available at `http://localhost:8080/api-docs/` .
  - Swagger Sample
- TCP API

How to use

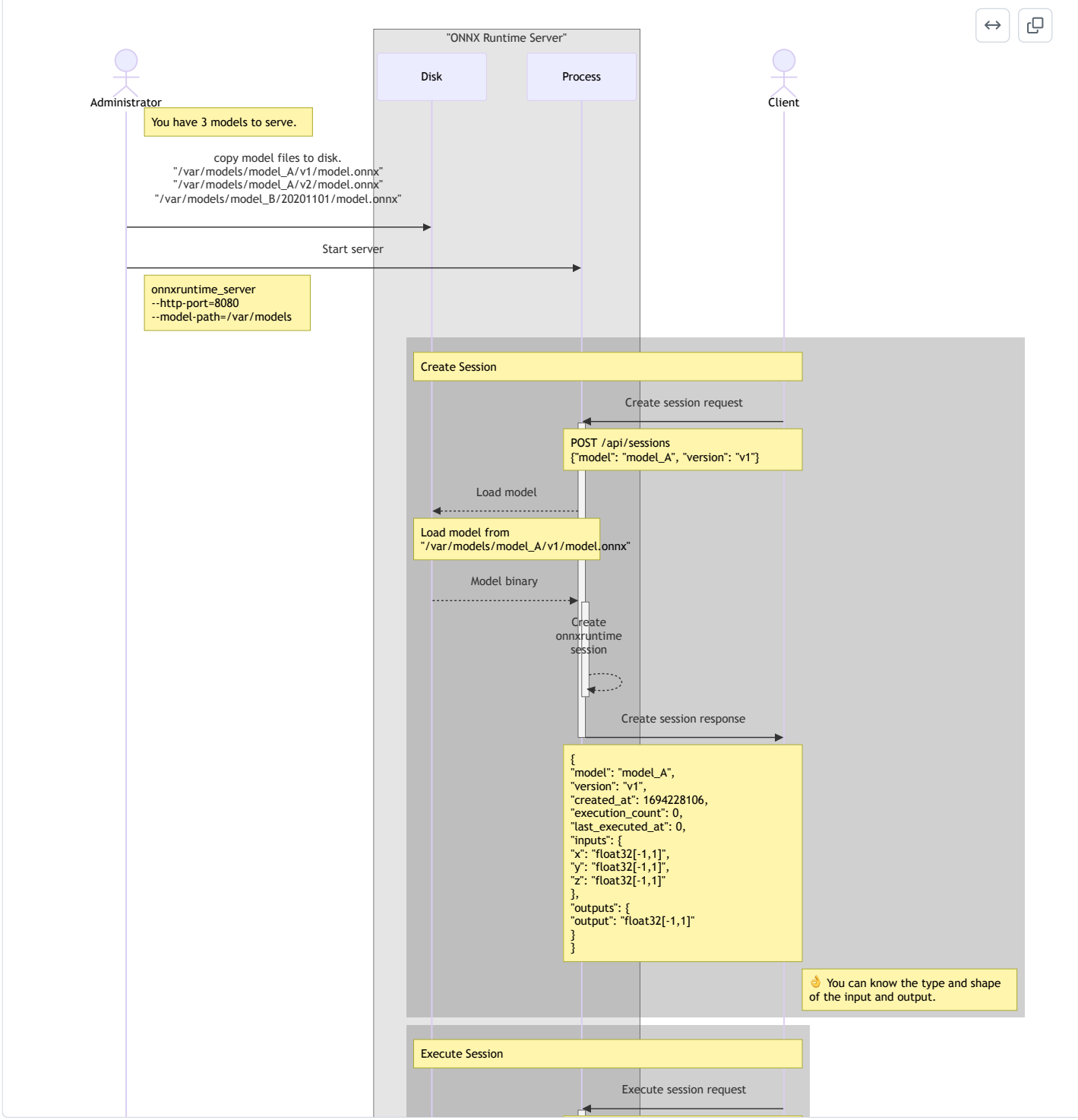
- A few things have been left out to help you get a rough idea of the usage flow.

Simple usage examples

Example of creating ONNX sessions at server startup



Example of the client creating and running ONNX sessions



Releases 5

v1.21.0 Latest
last month

+ 4 releases

Contributors 2

kibae Kibae Shin

mek-yt mekyt

Deployments 107

github-pages last month

+ 106 deployments

Languages